

中山大學
SUN YAT-SEN UNIVERSITY

数字图像处理 | 第十章实验

Otsu 图像阈值分割实验

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

课 程 教 师: 骆伟祺

起 始 日 期: 2025 年 12 月 23 日

结 束 日 期: 2025 年 12 月 24 日

实 验 地 点: B205

目录

1 实验简述 **2**

2 理论知识回顾 **2**

2.1 全局阈值处理 2

2.2 Otsu 方法（最大类间方差法） 2

3 代码设计与实现 **2**

4 实验结果与分析 **4**

4.1 直方图分析 4

4.2 分割结果 4

5 实验总结 **5**

1　实验简述

本次实验旨在掌握基本的图像分割技术，特别是基于阈值的分割方法。实验的核心任务是编写一个全局阈值化程序，利用 **Otsu 方法（最大类间方差法）** 自动估计最佳阈值，对包含目标物体的灰度图像进行二值化分割。

实验具体要求如下：

1. 编写一个全局阈值化程序，该程序使用 Otsu 方法自动估计阈值，程序的输出应该是一个二值分割图像。
2. 下载 origin.png 图像，并使用阈值程序对对象进行分割。

2　理论知识回顾

2.1　全局阈值处理

阈值处理是图像分割中最常用的方法之一。对于一幅灰度图像 $f(x,y)$，全局阈值处理通过选取一个阈值 T，将图像转换为二值图像 $g(x,y)$：

$$g(x,y)=\begin{cases}1\text{ 或 }255, & \text{if } f(x,y)>T\\ 0, & \text{if } f(x,y)\le T\end{cases} \tag{1}$$

其中，像素值大于 T 的被归类为前景（目标），否则归类为背景。

2.2　Otsu 方法（最大类间方差法）

Otsu 方法是一种自适应的阈值确定方法，它不需要人为设定参数，而是通过统计特性自动寻找最佳阈值。其核心思想是：** 最佳阈值应该使得分割后的前景和背景之间的差异最大（即类间方差最大）。**

假设图像像素被阈值 k 分为两类 C_1（背景）和 C_2（前景）。定义类间方差 $\sigma_B^2(k)$ 为：

$$\sigma_B^2(k)=P_1(k)[m_1(k)-m_G]^2+P_2(k)[m_2(k)-m_G]^2 \tag{2}$$

其中：

- $P_1(k),P_2(k)$ 分别为像素被分为 C_1 和 C_2 的概率。
- $m_1(k),m_2(k)$ 分别为 C_1 和 C_2 类像素的平均灰度值。
- m_G 为整个图像的全局平均灰度。

Otsu 算法遍历所有可能的灰度级 k（0-255），找到使 $\sigma_B^2(k)$ 最大的 k 值作为最佳阈值 k^*。

3　代码设计与实现

本实验的程序设计遵循标准的图像分割处理流程，旨在从含噪图像中准确提取目标区域。整体处理逻辑分为四个主要步骤，具体设计如下：

1. **图像读取与灰度化（Image Acquisition & Grayscale Conversion）：**程序首先读取输入图像 `origin.png`。由于 Otsu 算法是基于图像的亮度直方图进行统计分析的，因此必须将图像加载为单通道灰度模式（Grayscale）。这一步减少了计算维度，为后续处理提供基础数据。

2. **高斯预处理（Gaussian Smoothing）：**观察原始图像可知，图像中存在明显的颗粒噪声（椒盐噪声或高斯噪声）。若直接进行阈值分割，噪声点会被误判为前景或背景，导致分割边缘粗糙甚至产生孔洞。因此，本实验在阈值化之前，采用 5×5 的高斯核对图像进行平滑滤波。该步骤能够有效抑制高频噪声，使图像的灰度直方图分布更加平滑，有助于 Otsu 算法更准确地定位双峰之间的波谷。

3. **Otsu 阈值计算与分割（Thresholding）：**利用 OpenCV 的 `cv2.threshold` 函数，并启用 `cv2.THRESH_OTSU` 标志。算法自动遍历所有可能的灰度级（0-255），计算前景与背景的类间方差（Inter-class Variance）。程序选取使类间方差最大的灰度值作为全局最佳阈值 T^*，并将图像中灰度值大于 T^* 的像素置为 255（白色），其余置为 0（黑色）。

4. **结果可视化与持久化（Visualization & Output）：**程序最后将处理得到的二值分割图像保存为无损 PNG 格式，以供后续分析。同时，程序绘制并保存带有最佳阈值标记的灰度直方图，以便直观验证 Otsu 算法选取的阈值是否位于双峰谷底。

基于上述设计思路，具体的 Python 代码实现如下：

Listing 1: Otsu 阈值分割完整实现代码

```python
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

# 1. 自动获取路径
current_dir = os.path.dirname(os.path.abspath(__file__))
image_path = os.path.join(current_dir, 'origin.png')

# 2. 读取图像
img = cv2.imread(image_path, 0) # 0 表示以灰度模式读取

if img is None:
    print("Error: 无法读取图像")
else:
    # 3. 图像预处理
    # 原图含有噪声，使用 5x5 的高斯核进行平滑处理
    # 这一步能显著改善 Otsu 算法在直方图双峰不明显时的表现
    blur = cv2.GaussianBlur(img, (5, 5), 0)

    # 4. Otsu 阈值化
    # cv2.THRESH_OTSU 标志位让 OpenCV 自动寻找最佳阈值
    # ret 为计算得到的最佳阈值，th 为分割后的二值图像
    ret, th = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

    print(f"Otsu 算法计算出的最佳阈值为: {ret}")

    # 5. 保存结果
    # 保存二值化结果图
    cv2.imwrite(os.path.join(current_dir, 'otsu_result.png'), th)

    # 绘制并保存直方图
```

```
    plt.figure()
    plt.hist(blur.ravel(), 256, [0, 256], color='gray')
    plt.axvline(x=ret, color='r', linestyle='dashed', linewidth=2, label=f'Threshold: {ret}')
    plt.title('Histogram & Otsu Threshold')
    plt.legend()
    plt.savefig(os.path.join(current_dir, 'otsu_histogram.png'))
    plt.show()
```

4　实验结果与分析

4.1　直方图分析

实验生成的灰度直方图如图 1 所示。

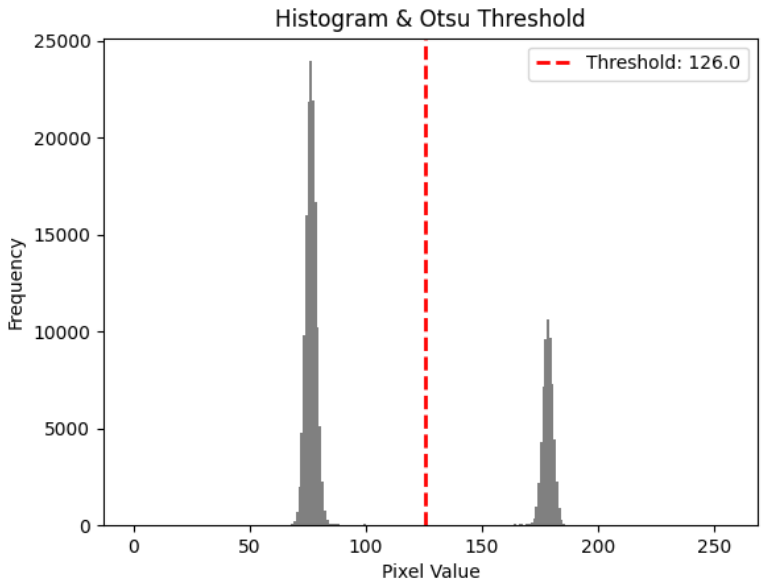
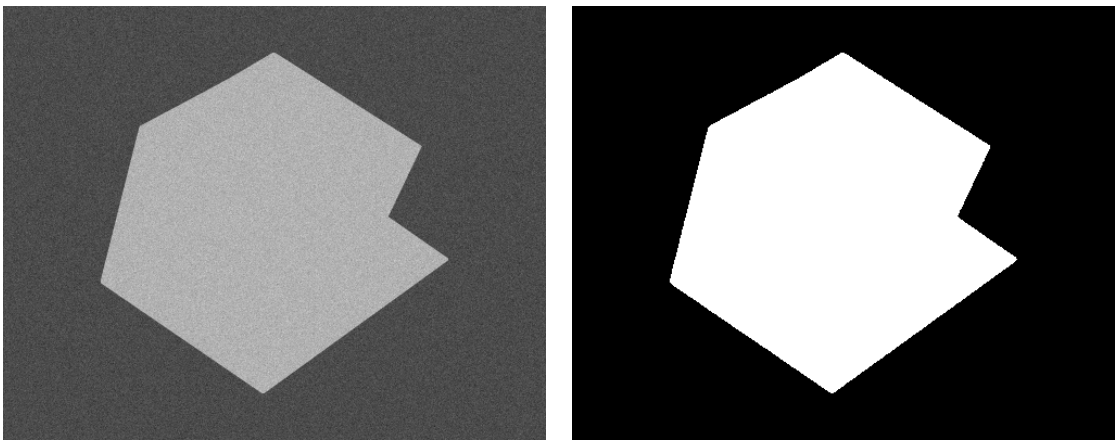


图 1: 图像灰度直方图与 Otsu 计算出的阈值

从直方图中可以观察到，经过高斯模糊处理后的图像直方图呈现出明显的 ** 双峰特性（Bimodal）**。左侧波峰代表背景像素的分布，右侧波峰代表目标物体的像素分布。Otsu 算法自动计算出的最佳阈值约为 **126**（图中红色虚线所示），该阈值精确地位于两个波峰之间的谷底位置，符合理论预期。

4.2　分割结果

最终的二值化分割结果如图 2 所示。



(a) 原始图像 (b) Otsu 分割结果

图 2: 分割前后对比

** 结果分析：**

- ** 完整性：** 目标多边形被完整地从背景中分离出来，内部填充均匀（白色，像素值 255），背景纯净（黑色，像素值 0）。
- ** 边缘平滑度：** 由于在阈值化前应用了 5×5 的高斯滤波，原始图像中的高频噪声（椒盐噪声）被有效抑制。如果直接对原图使用 Otsu，分割结果边缘可能会出现毛刺，或在背景中出现孤立的噪声点。

5 实验总结

本次实验成功实现了基于 Otsu 方法的图像自动阈值分割。

1. ** 掌握了 Otsu 算法原理 **：理解了如何通过最大化类间方差来自动寻找全局最佳阈值，解决了人工设定阈值的主观性问题。
2. ** 预处理的重要性 **：实验表明，对于含有噪声的图像，直接阈值化效果往往不佳。引入 ** 高斯模糊（Gaussian Blur）** 作为预处理步骤，能够平滑直方图，使双峰分离更明显，从而显著提高分割质量。
3. ** 工程实践能力 **：在代码实现中，解决了文件路径读取的常见问题，并规范了实验数据的保存格式，提升了代码的可移植性。